

CML 3 - Linear Regression deep dive

Notebook: CML3_HousePrice_LinearRegression.ipynb

Visuals: visuals/

Notation: we write parameters as θ (same idea as w / coef_ in sklearn).

1. Supervised learning setup

We are given a **training set** of m examples:

```
{ (x^(i), y^(i)) for i = 1, 2, ..., m }
```

- $x^{(i)}$ - input features (living area, bedrooms, income, ...)
- $y^{(i)}$ - target we want to predict (house price, ...)
- The superscript (i) is an **index** into the training set (not “to the power i ”)

One pair $(x^{(i)}, y^{(i)})$ is a **training example**.

Goal: learn a function (hypothesis) h so that $h(x)$ is a good predictor of y for new x .

```
Training set --> Learning algorithm --> hypothesis h
                                     |
                                     new x --> y_hat = h(x)
```

- If y is continuous (price, demand, temperature) -> **regression**
- If y takes discrete labels -> **classification** (later: logistic regression)

Linear regression = we restrict h to be a **linear** function of the features.

2. Linear hypothesis

One feature (living area x_1):

```
h_theta(x) = theta_0 + theta_1 * x1
```

Two features (area x_1 , bedrooms x_2):

```
h_theta(x) = theta_0 + theta_1 * x1 + theta_2 * x2
```

Intercept trick: introduce a constant feature $x_0 = 1$. Then pack everything into vectors:

```
h_theta(x) = theta_0*x0 + theta_1*x1 + ... + theta_n*xn
            = theta - x      (dot product)
```

Here n is the number of real features (not counting x_0).

Symbol	Meaning
θ_0	Intercept - prediction when all features are 0
θ_j ($j \geq 1$)	Weight on feature j
$h_{\theta}(x)$	Predicted value y_{hat}

Geometry

- 1 feature -> h is a **line** in the (x, y) plane
- Many features -> h is a **hyperplane** in feature space

Interpretation: θ_j is how much h changes when x_j increases by 1, holding other features fixed. That is an association in the fitted model - not automatically causal.

Visual: visuals/01_hypothesis.svg

3. Cost function - least squares

We want $h_{\theta}(x^{(i)})$ close to $y^{(i)}$ on the training set. Define the **cost**:

$$J(\theta) = (1/2) * \sum_{i=1..m} (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

The leading 1/2 is only for cleaner derivatives. Minimizing this J is the same as minimizing mean squared error (up to a positive constant).

Why square?

- Errors of either sign contribute positively
- Large mistakes are penalized **more** than small ones
- For linear h , J is a **convex** quadratic bowl (nice optimization landscape)

Learning problem: choose θ to **minimize** $J(\theta)$.

Visual: visuals/02_cost_mse.svg

Hygiene (from CML 2): minimize J on **train** only. Report MAE / RMSE / R^2 on a held-out test set.

4. Gradient descent / LMS

We rarely guess θ by hand. Use **gradient descent**: start from some initial θ , repeatedly step opposite the gradient of J :

$$\theta_j := \theta_j - \alpha * (\text{partial } J / \text{partial } \theta_j)$$

for all $j = 0, 1, \dots, n$ (update simultaneously)

- $\alpha > 0$ is the **learning rate** (step size)
- Too small -> slow crawl
- Too large -> can diverge or oscillate

4.1 Derivative for one training example

Temporarily drop the sum and take one example (x, y) :

$$J(\theta) = (1/2) * (h_{\theta}(x) - y)^2$$

Chain rule:

$$\begin{aligned} \text{partial } J / \text{partial } \theta_j &= (h_{\theta}(x) - y) * (\text{partial } / \text{partial } \theta_j)(h_{\theta}(x) - y) \\ &= (h_{\theta}(x) - y) * x_j \end{aligned}$$

because $h_{\theta}(x) = \sum_k \theta_k * x_k$.

So the update for one example is:

```
theta_j := theta_j + alpha * ( y - h_theta(x) ) * x_j
```

(This is the **LMS** / least-mean-squares update. Equivalently: subtract $\alpha * (h - y) * x_j$.)

Intuition

- Prediction already almost right -> tiny update
- Large error -> larger move of **theta**
- Feature x_j scales how much parameter j is adjusted

4.2 Batch gradient descent (full training set)

Put the sum back. The partial of J is:

```
partial J / partial theta_j
= sum_{i=1..m} ( h_theta(x^(i)) - y^(i) ) * x^(i)_j
```

Batch GD looks at **every** example on each step:

```
Repeat until convergence {
  for j = 0 .. n:
    theta_j := theta_j - alpha * sum_{i=1..m}
      ( h_theta(x^(i)) - y^(i) ) * x^(i)_j
  (use old theta to compute all updates, then apply them together)
}
```

Key fact: for linear regression, $J(\theta)$ is convex. With a not-too-large **alpha**, batch GD converges to the **global** minimum.

4.3 Stochastic (incremental) gradient descent

Instead of summing over all m examples each step, update using **one** example at a time (often shuffle the training set each pass):

```
Loop {
  for i = 1 .. m:
    for j = 0 .. n:
      theta_j := theta_j + alpha * ( y^(i) - h_theta(x^(i)) ) * x^(i)_j
}
```

- Cheaper per update; noisier path toward the minimum
- Useful when m is very large

5. Feature scaling and mean normalization

Different features often live on wildly different scales (income ~ few units, **total_rooms** ~ thousands). That:

- Makes the level sets of J **elongated** -> GD zig-zags and needs a tiny **alpha**
- Makes raw $|\theta_j|$ hard to compare across features

Standardization (common practice):

```
x_j <- (x_j - mu_j) / sigma_j
```

(or divide by the range $x_{j,\max} - x_{j,\min}$). After this, features are roughly zero-mean and comparable in scale.

CML 2 rule: compute μ_j , σ_j on **train only**, then transform train and test with those same statistics.

Visual: [visuals/04_scaling_lr.svg](#)

Even with a closed-form OLS solver (next section), scaling still helps **interpret** coefficients and keeps the Pipeline ready for iterative methods.

6. Normal equation (closed form)

Instead of iterating, the minimizer of J for linear least squares has a **closed form**.

Stack training inputs into a design matrix X (shape $m \times (n+1)$, each row is one $x^{(i)}$ including $x_0 = 1$) and targets into a vector y (length m). Then:

```
theta* = (X^T X)^(-1) X^T y
```

(when $X^T X$ is invertible).

Closed form	Gradient descent
One shot (matrix solve)	Many cheaper steps
Needs $X^T X$ well-behaved	Works when n is huge / streaming data
<code>sklearn.linear_model.LinearRegression</code>	Explicit loops / <code>SGDRegressor</code>

Practical notes

- If features are linearly dependent, $X^T X$ is singular $\rightarrow$ use a pseudoinverse or **regularization** (Ridge later)
- `sklearn`'s `LinearRegression` uses an efficient least-squares solver (OLS) - not a from-scratch GD loop

Visual: [visuals/03_closed_form.svg](#)

7. What we report after fitting

Training minimized squared error. For communication on **holdout** data:

Metric	Idea	Read as
MAE	mean of $ y_{\text{hat}} - y $	typical absolute miss (dollar-friendly)
RMSE	$\sqrt{\text{mean of } (y_{\text{hat}} - y)^2}$	same units as y ; heavier on large misses
R^2	$1 - \text{SSE} / \text{SST}$	fraction of variance explained (1 = perfect)

More precisely:

$$\begin{aligned}\text{MAE} &= (1/m) * \sum |y_{\text{hat}_i} - y_i| \\ \text{RMSE} &= \text{sqrt}((1/m) * \sum (y_{\text{hat}_i} - y_i)^2) \\ R^2 &= 1 - \frac{\sum (y_{\text{hat}_i} - y_i)^2}{\sum (y_{\text{bar}} - y_i)^2}\end{aligned}$$

Visual: visuals/05_metrics.svg

Useful diagnostic plots:

1. **Predicted vs actual** - points near the diagonal $y = x$
2. **Residuals** ($y_{\text{hat}} - y$) vs y_{hat} - cloud around 0 (assumptions session digs deeper)

8. End-to-end with sklearn (house prices)

Notebook: CML3_HousePrice_LinearRegression.ipynb

split -> impute / encode / scale (fit on train) -> LinearRegression -> test metrics + plot

```
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("prep", preprocess),          # ColumnTransformer from CML 2
    ("lr", LinearRegression()),    # OLS solver
])
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Try in the notebook

- Which scaled feature should dominate the coefficients? (often `median_income`)
- Edit the `my_house` feature dict -> model returns a dollar prediction

Same hypothesis h_{θ} and cost J - industrial solver instead of hand-written GD.

9. Where this sits in the course

Idea	This lecture	Later
Supervised setup, $h_{\theta} = \theta \cdot x$	§§1-2	-
Cost $J(\theta)$ (least squares)	§3	-
Batch & stochastic GD / LMS	§4	Session 5 deepens GD practice
Feature scaling	§5	Habit from CML 2
Normal equation	§6	-
Residual diagnostics / assumptions	light via plots	Session 4
Regularization	mentioned	Session 7

10. Close

- Assignment: Hire 477525
- Homework: Hire 477526
- Next: **Linear Regression assumptions** - when the least-squares story is misspecified, and how residuals tell you

Check yourself

1. Write `h_theta(x)` with the `x0 = 1` convention.
2. Write one step of batch GD for `theta_j`.
3. Why does a too-large learning rate break GD?
4. State the normal equation and one reason you might prefer GD anyway.